EXP NO: 2	SUPPORT VECTOR MACHINE (SVM) AND RANDOM FOREST FOR BINARY & MULTICLASS CLASSIFICATION

AIM

To build classification models using **Support Vector Machines (SVM)** and **Random Forest**, apply them to a dataset, and evaluate the models using performance metrics like accuracy and confusion matrix.

ALGORITHM

Part A: SVM Model

1. Import necessary libraries
2. Load and explore the dataset
3. Handle missing values if any
4. Encode categorical variables
5. plit dataset into training and testing sets
6. Build SVM classifier using SVC()
7. Train and predict
8. Evaluate the model using accuracy and confusion matrix

Part B: Random Forest Model

1. Initialize Random Forest using RandomForestClassifier()
2. Train and predict
3. Evaluate and compare with SVM

CODE:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# -------------------------------------------------------
# Upload Dataset
# -------------------------------------------------------

from google.colab import files
```

```
uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("Dataset Shape:", df.shape)
print(df.head())

# ------------------------------------------------------
# Handle Missing Values
# ------------------------------------------------------

for col in df.columns:
    if df[col].dtype == "object":
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].mean())

# ------------------------------------------------------
# Convert Categorical Columns into Numbers
# ------------------------------------------------------

encoder = LabelEncoder()

for col in df.columns:
    if df[col].dtype == "object":
        df[col] = encoder.fit_transform(df[col].astype(str))

# ------------------------------------------------------
# Select Target Column
# (Last column is considered as target)
# ------------------------------------------------------

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

print("\nFeatures:", X.columns.tolist())
print("Target:", df.columns[-1])

# ------------------------------------------------------
# Split Dataset
# ------------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# ------------------------------------------------------
# Feature Scaling
# ------------------------------------------------------

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ======================================================
# Support Vector Machine
# ======================================================

print("\n==============================")
print("Support Vector Machine Results")
print("==============================")

svm = SVC(kernel='rbf')

svm.fit(X_train, y_train)

svm_pred = svm.predict(X_test)

print("Accuracy:", accuracy_score(y_test, svm_pred))

print("\nClassification Report")
print(classification_report(y_test, svm_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, svm_pred))

# ======================================================
# Random Forest
# ======================================================

print("\n==============================")
print("Random Forest Results")
print("==============================")

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, rf_pred))

print("\nClassification Report")
print(classification_report(y_test, rf_pred))
```

```
print("\nConfusion Matrix")
print(confusion_matrix(y_test, rf_pred))

# ------------------------------------------------------
# Compare Models
# ------------------------------------------------------

print("\n==============================")
print("Model Comparison")
print("==============================")

print("SVM Accuracy          :", accuracy_score(y_test, svm_pred))
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
```

OUTPUT:

```
==============================
Support Vector Machine Results
==============================
Accuracy: 0.6372029665506281
```

```
    accuracy                           0.64      6607
   macro avg       0.00      0.01      0.00      6607
weighted avg       0.54      0.64      0.58      6607


Confusion Matrix
[[ 221    0    0 ...    0    0    0]
 [   3    0    0 ...    0    0    0]
 [   2    0    0 ...    0    0    0]
 ...
 [   0    0    0 ...    0    1    0]
 [   0    0    0 ...    0 3142    0]
 [   0    0    0 ...    0    0  735]]
```

```
==================================
Random Forest Results
==================================
/usr/local/lib/python3.12/dist-packag
  _warn_prf(average, modifier, f"{met
/usr/local/lib/python3.12/dist-packag
  _warn_prf(average, modifier, f"{met
/usr/local/lib/python3.12/dist-packag
  _warn_prf(average, modifier, f"{met
/usr/local/lib/python3.12/dist-packag
  _warn_prf(average, modifier, f"{met
/usr/local/lib/python3.12/dist-packag
  _warn_prf(average, modifier, f"{met
/usr/local/lib/python3.12/dist-packag
  _warn_prf(average, modifier, f"{met
Accuracy: 0.6652035719691236
```

```
    accuracy                           0.67      6607
   macro avg       0.06      0.06      0.05      6607
weighted avg       0.66      0.67      0.66      6607


Confusion Matrix
[[ 221    0    0 ...    0    0    0]
 [   0    0    1 ...    0    0    0]
 [   0    0    1 ...    0    0    0]
 ...
 [   0    0    0 ...    0    0    0]
 [   0    0    0 ...    0 3146    0]
 [   0    0    0 ...    0    0  735]]
```

```
==============================
Model Comparison
==============================
SVM Accuracy          : 0.6372029665506281
Random Forest Accuracy: 0.6652035719691236
```

RESULT:

The Support Vector Machine (SVM) and Random Forest algorithms were successfully implemented for both binary and multiclass classification tasks. The models were trained and tested on the given dataset, and both achieved good accuracy.